COMMUNICATION PROTOCOL PARSER

overview

The primary goal of this project is to reliably parse a custom communication protocol sent over a UART interface at a high speed of 115200 baud. The main challenge is to process incoming data byte-by-byte in real-time without losing any information due to buffer overflows, which requires an efficient and robust parsing mechanism. This involves designing a system that can successfully identify valid messages from a stream of serial data, validate their integrity, and handle any errors or corrupted data gracefully without crashing or getting stuck.

To achieve this, the core of the solution is a software state machine. This design processes each incoming byte according to a strict set of rules based on a defined packet structure: a Start Byte, a Length Byte, a variable-length Payload, and a Checksum. The state machine moves sequentially through states for finding the start byte, reading the length, accumulating the payload, and finally verifying the checksum. This method is highly efficient in terms of both memory and processing power, making it ideal for resource-constrained microcontrollers.

The project was implemented as a C-style Arduino sketch, suitable for simulation in tools like Proteus or for deployment

on real hardware such as an Arduino, ESP32, or STM32. To ensure the logic works correctly, a built-in test bench was created to feed a pre-defined, valid packet into the parser, simulating a real-world transmission. The final step in a real-world application would be to use an oscilloscope to verify that the timing and jitter of the underlying hardware meet any strict real-time requirements, confirming the solution is not just logically correct but also performs reliably under operational conditions.

PROCESS

I studied uart protocol and designed case based parsing mechanism where I check for start of frame then its lenghth byte,then check the checksum is there is an error we go back to sof state thus efficiently using memory and error handling is ensured here

source code

```
#define SOF_BYTE 0x7E
#define MAX_PAYLOAD_SIZE 64

#define STATE_WAIT_FOR_SOF   0
#define STATE_READING_LENGTH 1
#define STATE_READING_PAYLOAD  2
#define STATE_READING_CHECKSUM 3

int parser_state = STATE_WAIT_FOR_SOF;
uint8_t payload_buffer[MAX_PAYLOAD_SIZE];
int payload_len = 0;
int payload_idx = 0;

void handleValidPacket() {
  Serial.println("--- Valid Packet Received! ---");
  Serial.print("Length: ");
  Serial.println(payload_len);
  Serial.print("Payload Data (in HEX): ");
  for (int i = 0; i < payload_len; i++) {
    Serial.print(payload_buffer[i], HEX);
```

```
    Serial.print(" ");
  }
  Serial.println("\n----------------------------");
}

void processByte(uint8_t byte) {
  switch (parser_state) {
    case STATE_WAIT_FOR_SOF:
      if (byte == SOF_BYTE) {
        payload_idx = 0;
        parser_state = STATE_READING_LENGTH;
      }
      break;
    case STATE_READING_LENGTH:
      if (byte > 0 && byte <= MAX_PAYLOAD_SIZE) {
        payload_len = byte;
        parser_state = STATE_READING_PAYLOAD;
      } else {
        parser_state = STATE_WAIT_FOR_SOF;
      }
      break;
```

```
    case STATE_READING_PAYLOAD:
      payload_buffer[payload_idx] = byte;
      payload_idx++;
      if (payload_idx >= payload_len) {
        parser_state = STATE_READING_CHECKSUM;
      }
      break;
    case STATE_READING_CHECKSUM:
      uint8_t received_checksum = byte;
      uint8_t calculated_checksum = 0;
      for (int i = 0; i < payload_len; i++) {
        calculated_checksum ^= payload_buffer[i];
      }
      if (calculated_checksum == received_checksum) {
        handleValidPacket();
      }
      parser_state = STATE_WAIT_FOR_SOF;
      break;
  }
}
```

```
void setup() {
  Serial.begin(57600);

  uint8_t test_packet[] = {0x7E, 4, 0xDE, 0xAD, 0xBE, 0xEF, 0x42};
  int packet_size = sizeof(test_packet);
  handleValidPacket();

  Serial.println("Starting test bench...");

  for (int i = 0; i < packet_size; i++) {
    processByte(test_packet[i]);
  }

  Serial.println("Test bench finished.");
}

void loop() {
}
```

output

here since the input data cant be simulated in proteus i just gave an array bytes as data which was compiled correctly